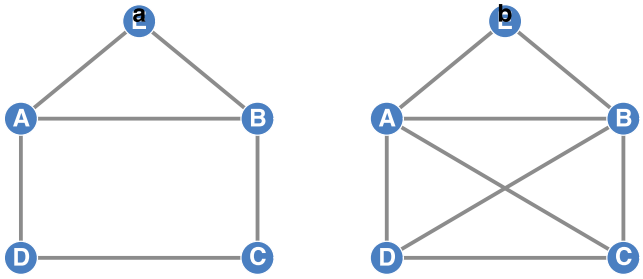
Exercice formatif : introduction aux réseaux en biologie — corrigé

Chaque réponse est suivie de la notion à revoir si elle a été manquée.

1. Sans lever le crayon



(a) et (b) : peut-on parcourir chaque lien une seule fois, sans lever le crayon ? Si oui, d'où faut-il partir ? Justifiez sans tracer. (c) Un réseau de régulation (un facteur de transcription → ses gènes cibles) est-il dirigé ? Peut-il contenir des boucles ? Que signifierait une boucle ?

On compte les degrés impairs. (a) Degrés A 3, B 3, C 2, D 2, E 2 : **deux** nœuds impairs (A et B), donc oui, en partant de A et en finissant en B (ou l'inverse). (b) Degrés A 4, B 4, C 3, D 3, E 2 : deux impairs (C et D), donc oui, en partant de C ou de D. (c) **Dirigé** (facteur → cible) ; une boucle est une **autorégulation** : le facteur régule son propre gène. *À revoir : le théorème d'Euler, les types de graphes (G-GRA).*

2. Construire et lire un graphe

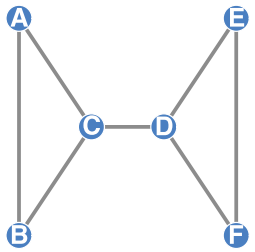
```
aretes <- tibble(from  = c("P1", "P1", "P2", "P3"),
                 to    = c("P2", "P3", "P3", "P4"),
                 score = c(0.9, 0.4, 0.7, 0.95))
proteines <- tibble(name = c("P1", "P2", "P3", "P4", "P5"))
g1 <- graph_from_data_frame(aretes)
g2 <- graph_from_data_frame(aretes, directed = FALSE, vertices = proteines)
```

(a) Que contient la première ligne de `g1` (lettres et nombres) ? Pourquoi pas de `W` ? (b) Et celle de `g2` ? (c) Écrivez la matrice d'adjacence de `g2`. (d) Déduisez-en le degré de chaque protéine et la densité de `g2`.

```
#> IGRAPH 7fe4738 DN-- 4 4 --
#> IGRAPH 70cd04b UN-- 5 4 --
#>    P1 P2 P3 P4 P5
#> P1  0  1  1  0  0
#> P2  1  0  1  0  0
#> P3  1  1  0  1  0
#> P4  0  0  1  0  0
#> P5  0  0  0  0  0
```

(a) `DN--` : **dirigé** (défaut de `graph_from_data_frame()`), nommé, 4 nœuds, 4 liens. Pas de `W` : l'attribut s'appelle `score`, pas `weight`. (b) `UN--`, **5** nœuds : P5, sans aucun lien, n'existe que grâce à `vertices`. (c) Ci-dessus : symétrique, diagonale nulle, ligne et colonne de P5 à 0. (d) Somme des lignes : P1 2, P2 2, P3 3, P4 1, P5 0 ; densité 4 / (5 × 4 / 2) = **0,4**. *À revoir : lire l'en-tête, nœuds isolés, matrice d'adjacence (G-REP).*

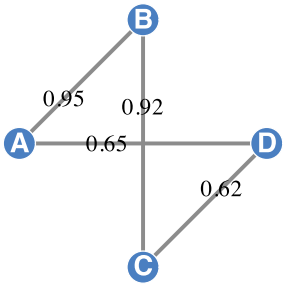
3. Calculer à la main



Calculez (a) le degré de chaque nœud et la densité ; (b) la distance de A à F et le diamètre ; (c) le *clustering* local de A et de C ; (d) quels nœuds ont la plus forte intermédiarité, et pourquoi ?

(a) A 2, B 2, C 3, D 3, E 2, F 2 ; densité 7 / 15 = **0,47**. (b) A–C–D–F : **3** ; diamètre **3**. (c) A : voisins B et C, liés : **1**. C : voisins A, B, D ; un seul lien (A–B) sur 3 possibles : **0,33**. (d) **C et D** (6 chacun, les autres 0) : tous les chemins entre les deux triangles passent par le pont C–D. *À revoir : degré, distance, clustering, intermédiarité (G-TOP).*

4. Des poids qui trompent



Dans ce petit réseau de co-expression, l'attribut `weight` est la corrélation entre deux gènes. (a) Quel chemin de A à C `shortest_paths(w, "A", "C")` renvoie-t-il ? Calculez sa longueur. (b) Quel gène a la plus forte intermédiarité selon `betweenness(w)` ? Et sans les poids ? (c) Proposez deux corrections.

(a) igraph lit `weight` comme une **longueur** : A–B–C = 0.95 + 0.92 = 1.87, A–D–C = 0.65 + 0.62 = 1.27. Il renvoie **A–D–C**, le chemin des corrélations les **plus faibles**. (b) Avec les poids : A 0, B 0, D 1, C 1 (D porte le chemin A–C). Sans les poids : A 0.5, B 0.5, D 0.5, C 0.5, tous égaux. Un classement faux, **sans message**. (c) `weights = NA` (nombre de liens), ou une vraie distance `weights = 1 / E(w)$weight` (le chemin devient A–B–C). *À revoir : piège des poids lus comme des distances (G-TOP, G-AUD).*

5. Un réseau de co-expression

```
#>        DUSP1 ZBTB16 KCTD12 DDX3Y
#> DUSP1   1.00   0.98  -1.00 -0.04
#> ZBTB16  0.98   1.00  -0.98 -0.14
#> KCTD12 -1.00  -0.98   1.00  0.01
#> DDX3Y  -0.04  -0.14   0.01  1.00
```

Corrélations de Pearson entre quatre gènes d'`airway` (8 échantillons) : *DUSP1* et *ZBTB16* sont induits par la dexaméthasone, *KCTD12* est réprimé, *DDX3Y* est sur le chromosome Y. (a) Quels liens avec le seuil r > 0.8 ? Quels gènes sont isolés ? (b) Et avec `abs(r) > 0.8` ? Qu'est-ce que cela change pour l'interprétation ? (c) Pourquoi faut-il se méfier d'une corrélation calculée sur 8 échantillons ? (d) Un lien *DUSP1–ZBTB16* veut-il dire que l'un régule l'autre ?

(a) 1 lien : DUSP1–ZBTB16 ; isolés : *KCTD12*, *DDX3Y*. (b) 3 liens : DUSP1–ZBTB16, DUSP1–KCTD12, ZBTB16–KCTD12. Les gènes **anticorrélés** sont liés : induits et réprimés se retrouvent dans le même module, qui ne dit plus dans quel sens varient ses gènes (réseau non signé). (c) Avec si peu d'échantillons, de fortes corrélations apparaissent **par hasard** : sur 150 gènes de bruit pur et 8 échantillons, 93 paires sur 11 175 dépassent r = 0.8 (diapo « Piège : 8 échantillons »). (d) Non : une corrélation n'a ni **sens** ni **causalité** ; les deux gènes peuvent répondre au même facteur (le récepteur des glucocorticoïdes) : un lien **indirect**. *À revoir : co-expression, seuil, inférence (G-BIO).*

6. Interpréter des modules

On applique `cluster_louvain()` (avec `set.seed(1)`) au réseau de co-expression des 150 gènes d'`airway` (r > 0.8), puis on compte les gènes de chaque module selon leur statut et leur chromosome (Y ou autre) :

```
#>
#> module induit réprimé non DE, autre non DE, chr. Y
#>      1      0       0             2             7
#>      2      0      17             1             0
#>      3      6       4            26             0
#>      4     45       0             0             0
#>      5     26       0             3             0
#>      6      1       3             6             0
#>      7      0       1             0             0
#>      8      0       0             1             0
#>      9      0       0             1             0
```

(a) Que montrent les modules induits et réprimés ? (b) Interprétez le module 1. Comment le vérifier ? (c) Une camarade relance le calcul avec une autre graine : son module 1 contient des gènes induits. A-t-elle fait une erreur ?

(a) Les modules sont presque **purs** : des modules de gènes induits, un module de gènes réprimés. Le traitement structure fortement l'expression. (b) Le module 1 regroupe des gènes **non DE**, presque tous sur le **chromosome Y** : ils varient ensemble parce qu'un des donneurs est une **donneuse** (gènes du Y éteints, *XIST* exprimé). Un module réel, sans rapport avec la dexaméthasone ; on le vérifie en regardant l'expression de *RPS4Y1* ou *XIST* par échantillon, ou les métadonnées des donneurs. (c) Pas forcément : Louvain est **aléatoire**, et surtout les numéros de modules sont des **éti-**

quettes ; il faut comparer les modules par leur **contenu**. *À revoir : modules, Louvain, interprétation (G-MOD).*

7. Vérifier une analyse

Un étudiant analyse un réseau d'interactions protéiques téléchargé de STRING :

```
ppi <- read_tsv("interactions.tsv")   # prot1, prot2, weight = score STRING (0 à 1)
g <- graph_from_data_frame(ppi)
diameter(g)                           # « le signal traverse la cellule en ... étapes »
modules <- cluster_louvain(g)
```

(a) R s'arrête à la dernière ligne avec `Multi-level community detection works for undirected graphs only`. Pourquoi ? (b) Deux autres problèmes, que R ne signale pas. (c) Après correction, il écrit : « *TP53* est le plus grand hub du réseau, c'est donc la protéine la plus importante ». Une objection.

(a) `graph_from_data_frame()` crée un graphe **dirigé** par défaut ; une interaction protéique n'a pas de sens : `directed = FALSE`. Ici l'erreur est visible ; ailleurs (degrés, chemins), elle passe en silence. (b) `diameter()` utilise `weight` comme une **distance** : le score de confiance est lu à l'envers (`weights = NA`). Les protéines sans interaction au-dessus du seuil de STRING sont **absentes** du graphe. Louvain est aléatoire : fixer la graine. (c) *TP53* est l'un des gènes les plus **étudiés** : il a plus d'interactions **connues**, pas forcément plus d'interactions réelles ; un hub issu de la littérature est d'abord un gène beaucoup publié. *À revoir : vérifier une analyse de réseau (G-AUD), les hubs (G-TOP).*